

NCBA Country Info Service — Kubernetes Deployment Guide

Step 9 — Deploying the application on Kubernetes

NCBA Integration Microservices

Kubernetes Deployment Guide

This guide describes how to deploy the **Country Info integration microservice** (Spring Boot + MySQL) onto a Kubernetes cluster using the manifests under the `k8s/` directory.

1. Overview

The deployment consists of:

Component	Kind	File	Purpose
Namespace	Namespace	namespace.yaml	Isolates all resources under ncba-countryinfo
App config	ConfigMap	configmap.yaml	Non-sensitive config (DB host/port/name, SOAP endpoint, timeouts)
Credentials	Secret	secret.yaml	DB username/password and MySQL root password
Database	StatefulSet + headless Service + PVC	mysql-deployment.yaml	MySQL 8 with persistent storage
Application	Deployment	deployment.yaml	2 replicas, probes, resource limits, rolling updates
App network	Service (ClusterIP)	service.yaml	Stable in-cluster endpoint (port 80 → 8080)
Ingress	Ingress	ingress.yaml	External host-based routing via NGINX
Autoscaling	HorizontalPodAutoscaler	hpa.yaml	Scales app 2→10 pods at 70% CPU

```

      Ingress(nginx)      Service(ClusterIP)      Deployment (HPA 2..10)
client → countryinfo.local → countryinfo-app:80 → pod:8080 pod:8080 ...
                                         |
                                         v
                                StatefulSet: mysql (headless svc + PVC)
                                         |
                                         v SOAP (egress)
                                http://webservices.oorsprong.org/.../CountryInfoService.wso

```

2. Prerequisites

- A running Kubernetes cluster (minikube, kind, or a managed cluster) and `kubectl` configured to talk to it (`kubectl cluster-info`).
 - An **ingress controller** (e.g. ingress-nginx) installed — required only if you use `ingress.yaml`.
 - **metrics-server** installed — required for the HorizontalPodAutoscaler to read CPU metrics (`kubectl top pods` must work).
 - A container registry the cluster can pull from, or a local image loaded into the cluster (minikube/kind).
-

3. Build and publish the image

The manifest `k8s/deployment.yaml` references the image `country-info-service:1.0.0`. Build it and make it available to the cluster.

Option A — minikube (use the cluster's Docker daemon)

```
eval $(minikube docker-env)
docker build -t country-info-service:1.0.0 .
# Image is now visible to the cluster; keep imagePullPolicy: IfNotPresent
```

Option B — kind (load the image into the cluster)

```
docker build -t country-info-service:1.0.0 .
kind load docker-image country-info-service:1.0.0
```

Option C — a real registry

```
docker build -t <registry>/country-info-service:1.0.0 .
docker push <registry>/country-info-service:1.0.0
# Then update the image: field in k8s/deployment.yaml to <registry>/country-info-
service:1.0.0
```

4. Deploy — step by step

Apply the manifests in dependency order. Each step includes a verification.

4.1 Namespace

```
kubectl apply -f k8s/namespace.yaml
kubectl get namespace ncba-countryinfo
```

4.2 Configuration and secrets

```
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret.yaml

kubectl -n ncba-countryinfo get configmap countryinfo-config -o yaml
kubectl -n ncba-countryinfo get secret countryinfo-secret
```

Production note: do not commit real Secret manifests. Use Sealed Secrets, the External Secrets Operator, or HashiCorp Vault. The provided `secret.yaml` uses `stringData` for readability during the assessment.

4.3 Database (MySQL StatefulSet)

```
kubectl apply -f k8s/mysql-deployment.yaml
kubectl -n ncba-countryinfo rollout status statefulset/mysql --timeout=180s

kubectl -n ncba-countryinfo get pods -l app=mysql
kubectl -n ncba-countryinfo get pvc          # mysql-data-mysql-0 should be Bound
```

Using an external MySQL instead? If the database lives outside the cluster (managed instance, VM, or your host machine), skip this StatefulSet and apply `k8s/external-mysql.yaml` instead — it maps the in-cluster name `mysql` to your external host via an `ExternalName` Service, so the app needs no changes. Put the real DB credentials in `secret.yaml`. See README §4 *Option D*.

4.4 Application

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl -n ncba-countryinfo rollout status deployment/countryinfo-app --timeout=180s

kubectl -n ncba-countryinfo get pods -l app=countryinfo-app
kubectl -n ncba-countryinfo get endpoints countryinfo-app # should list 2 pod IPs
```

4.5 Autoscaling and ingress

```
kubectl apply -f k8s/hpa.yaml
kubectl apply -f k8s/ingress.yaml

kubectl -n ncba-countryinfo get hpa
kubectl -n ncba-countryinfo get ingress
```

5. Verify the deployment end to end

5.1 Without an ingress (port-forward the Service)

```
kubectl -n ncba-countryinfo port-forward svc/countryinfo-app 8080:80
# In another terminal:
curl http://localhost:8080/actuator/health
curl -X POST http://localhost:8080/api/countries \
  -H 'Content-Type: application/json' -d '{"name":"kenya"}
```

5.2 With the ingress

```
# Map the ingress address to the configured host:
echo "$(kubectl -n ncba-countryinfo get ingress countryinfo-ingress -o
  jsonpath='{.status.loadBalancer.ingress[0].ip}') countryinfo.local" | sudo tee -
  a /etc/hosts

# (minikube: use `minikube ip`)

curl -X POST http://countryinfo.local/api/countries \
  -H 'Content-Type: application/json' -d '{"name":"kenya"}
```

5.3 Health and probe status

```
kubectl -n ncba-countryinfo get pods # all Running, READY 2/2 etc.
kubectl -n ncba-countryinfo exec deploy/countryinfo-app -- \
  wget -q0- http://localhost:8080/actuator/health/readiness
```

6. Configuration reference

The application reads configuration from environment variables, injected from the ConfigMap (non-sensitive) and Secret (credentials):

Variable	Source	Example
DB_HOST , DB_PORT , DB_NAME	ConfigMap	mysql , 3306 , countryinfo
SPRING_PROFILES_ACTIVE	ConfigMap	prod (JSON structured logs)
SOAP_ENDPOINT	ConfigMap	oorsprong CountryInfoService URL
SOAP_CONNECT_TIMEOUT_MS , SOAP_READ_TIMEOUT_MS	ConfigMap	5000 , 10000
DB_USERNAME , DB_PASSWORD	Secret	appuser , apppass

To change configuration, edit the ConfigMap/Secret and restart the rollout:

```
kubectl -n ncba-countryinfo apply -f k8s/configmap.yaml
kubectl -n ncba-countryinfo rollout restart deployment/countryinfo-app
```

7. Scaling, updates, and rollback

Manual scale:

```
kubectl -n ncba-countryinfo scale deployment/countryinfo-app --replicas=4
```

Autoscaling is handled by the HPA (min 2, max 10, target 70% CPU):

```
kubectl -n ncba-countryinfo get hpa countryinfo-app -w
```

Rolling update (new image) — `maxSurge: 1, maxUnavailable: 0` means no downtime; a new pod must pass readiness before an old one is removed:

```
kubectl -n ncba-countryinfo set image deployment/countryinfo-app \
  countryinfo-app=country-info-service:1.1.0
kubectl -n ncba-countryinfo rollout status deployment/countryinfo-app
```

Rollback to the previous revision:

```
kubectl -n ncba-countryinfo rollout undo deployment/countryinfo-app
kubectl -n ncba-countryinfo rollout history deployment/countryinfo-app
```

8. Teardown

```
# Remove app + db but keep the namespace:
kubectl -n ncba-countryinfo delete -f k8s/ingress.yaml -f k8s/hpa.yaml \
  -f k8s/service.yaml -f k8s/deployment.yaml -f k8s/mysql-deployment.yaml

# Or remove everything (also deletes PVCs/data):
kubectl delete namespace ncba-countryinfo
```

For runtime issues, see the companion **Kubernetes Troubleshooting Guide**.